

MAE 146: Intro to Machine Learning with Python

Homework #5

Prof. Michael Tolley

Assigned: May 15, 2026

Due: May 22, 2026

Instructions:

- Homework submission is through Gradescope. Please leave extra time for submission, you can always re-submit up until the deadline.
- Your solution should not just give an answer, you should demonstrate how you got the answer.
- **For analytical problems**, submit handwritten answers (no typed responses).
- **For programming problems**, append any Python code you have used to solve the problem to your homework, and show any inputs you gave to your code, and any outputs it generated. You could do this by writing your solution in a Jupyter notebook, or by copying the output from your IDE (like Spyder) which may be easier for debugging. Either way, please add a pdf of your combined solution to your Gradescope submission (there will not be a separate code submission for this assignment).
- If you ever think a problem is stated incorrectly, not enough information is given, or it is impossible to solve, don't panic! Just make a reasonable assumption that will allow you to solve the problem, clearly stating your assumption(s), or indicate why it is not possible to solve the problem.
- For any questions regarding the assignment, please attend office hours (listed in the syllabus), or post them to Piazza.
- For initial homework submissions, we will only check that you have made an honest effort for each problem (e.g., that you did not just repeat the problem statement).
- After submission, you will have the opportunity to correct your solutions, and submit your corrections with the next homework (e.g., attach your HW1 corrections to the end of your submission for HW2), using the posted solutions as a guide.
- The corrected previous week's assignment should show handwritten corrections on top of the original submission (as if you were giving feedback on your own homework), as well as corrected code/plots for coding problems (corrected code is not required if your initial submission was correct).
- You are encouraged to discuss the homework with classmates, however your written submissions should be your own work. Completing your own assignments is the best way to learn the material and prepare for exams!

Analytical Problems (24 points)

1. (6 points) **Kernel functions (I)** Answer the following:

- (a) (1 point) Assume the map from $\mathbf{x} \in \mathbb{R}^2$ to feature vectors $\phi(\mathbf{x})$ is given by $\phi(\mathbf{x}) = (x_1, x_2, x_1^2, \sqrt{2}x_1x_2, x_2^2)^\top \in \mathbb{R}^5$. Given another vector $\mathbf{x}'$ express the product $\phi(\mathbf{x}) \cdot \phi(\mathbf{x}')$ as a function of the inner product $\mathbf{x} \cdot \mathbf{x}'$.
- (b) (1 point) Which of the following feature vectors $\phi(\mathbf{x})$ produces the Kernel:

$$K(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x}) \cdot \phi(\mathbf{x}') = x_1x'_1 + x_2x'_2 + x_3x'_3 + x_2x'_3 + x_3x'_2$$

$$(1) \phi(\mathbf{x}) = (x_1, x_2, x_3)^\top \quad (2) \phi(\mathbf{x}) = x_1 + x_2 + x_3 \quad (3) \phi(\mathbf{x}) = (x_1, x_2 + x_3)^\top \quad (4) \phi(\mathbf{x}) = (x_1 + x_3, x_1 + x_2)^\top$$

Justify your answer.

- (c) (1 point) Recall that if $f : \mathbb{R}^d \rightarrow \mathbb{R}$ and $K(\mathbf{x}, \mathbf{x}')$ is a Kernel, then so is $\tilde{K}(\mathbf{x}, \mathbf{x}') = f(\mathbf{x})K(\mathbf{x}, \mathbf{x}')f(\mathbf{x}')$. If there is a $\phi(\mathbf{x})$ such that $K(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x}) \cdot \phi(\mathbf{x}')$, which of the following gives $\tilde{K}(\mathbf{x}, \mathbf{x}') = \varphi(\mathbf{x}) \cdot \varphi(\mathbf{x}')$?

$$(1) \varphi(\mathbf{x}) = f(\mathbf{x})K(\mathbf{x}, \mathbf{x}') \quad (2) \varphi(\mathbf{x}) = \phi(\mathbf{x})K(\mathbf{x}, \mathbf{x}') \quad (3) \varphi(\mathbf{x}) = f(\mathbf{x}) \quad (4) \varphi(\mathbf{x}) = f(\mathbf{x})\phi(\mathbf{x}).$$

Justify your answer.

- (d) (1 point) Let $\mathbf{x}, \mathbf{q} \in \mathbb{R}^2$ be two feature vectors, and let $K(\mathbf{x}, \mathbf{q}) = (\mathbf{x}^\top \mathbf{q} + 1)^2$, which is known as a polynomial Kernel. Assuming we can write $K(\mathbf{x}, \mathbf{q}) = \phi(\mathbf{x})^\top \phi(\mathbf{q})$, derive an expression for $\phi(\mathbf{x}) = (f_1(x_1, x_2), \dots, f_N(x_1, x_2))^\top \in \mathbb{R}^N$.

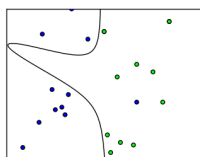
2. (6 points) **Kernel functions (II)** Consider fitting a kernelized large margin classifier to a dataset $(\mathbf{x}^{(i)}, y^{(i)})$ where $\mathbf{x}^{(i)} \in \mathbb{R}^2$ and $y^{(i)} \in \{-1, 1\}$ for all $i = 1, \dots, n$. In other words, this is a classification method by which one computes $\boldsymbol{\theta}$ and θ_0 to minimize the following objective:

$$L(\boldsymbol{\theta}, \theta_0) = \frac{1}{n} \sum_{i=1}^n \text{Loss}_h(y^{(i)}(\boldsymbol{\theta} \cdot \phi(\mathbf{x}^{(i)}) + \theta_0)) + \frac{\lambda}{2} \|\boldsymbol{\theta}\|^2,$$

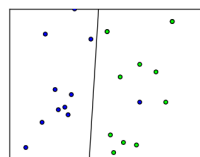
where ϕ is the feature vector associated with the kernel function, Loss_h is the hinge loss, and $\boldsymbol{\theta}$ is the feature vector associated with the kernel function.

Figures (a) to (d) show 4 different kernelized large-margin classifier models estimated from the same 11 data points. A different kernel is used to obtain each plot but you've forgotten which plot corresponds to which kernel.

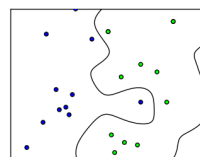
- (a) For each plot: Identify which kernel was used, out of: linear, quadratic, cubic, or RBF kernel
- (b) Overall, describe qualitatively how the resulting classifiers would vary with the parameter λ . I.e., if the value of λ were increased, would the model fit the training data better or worse? Justify your answer.



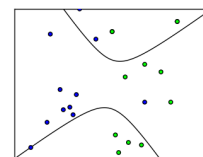
i Plot A



ii Plot B



iii Plot C



iv Plot D

3. (6 points) **Computation cost comparison of KPA and PA** Consider the quadratic feature transformation $\phi(\mathbf{x}) = [x_1, x_2, x_1^2, \sqrt{2}x_1x_2, x_2^2]^\top$, for 2D feature vectors $\mathbf{x} = (x_1, x_2)^\top \in \mathbb{R}^2$. Answer the following:

- (a) Suppose that $(\mathbf{x}, y)$ is a training example, with y a label, and $\boldsymbol{\theta}$ is a classifier vector. Assuming that $+$, $-$, $\times$ operations count as 1 each, how many computations in total do we have to perform in order to obtain $y(\boldsymbol{\theta} \cdot \phi(\mathbf{x}))$?
- (b) Recall that the Kernel of the quadratic feature transformation is $K(\mathbf{x}, \mathbf{x}') = \mathbf{x} \cdot \mathbf{x}' + (\mathbf{x} \cdot \mathbf{x}')^2$. Calculate the number of computations that you need to perform in order to evaluate the Kernel $K(\mathbf{x}, \mathbf{x}')$. To do this, first compute the number of computations to obtain $\mathbf{x} \cdot \mathbf{x}'$, then, using this value, compute the number of additional computations to obtain $K(\mathbf{x}, \mathbf{x}')$.
- (c) Suppose that there are only two training examples, $(\mathbf{x}, y)$ and $(\mathbf{x}', y')$ that you use to train a Perceptron algorithm. Without any Kernel trick, we would have to at least perform two classifications, $y(\boldsymbol{\theta} \cdot \phi(\mathbf{x}))$ and $y'(\boldsymbol{\theta} \cdot \phi(\mathbf{x}'))$, in the Perceptron loop, to decide whether or not to update the parameter $\boldsymbol{\theta}$. How many computations would these classifications require?
- (d) If we were to use the Perceptron algorithm with a Kernel trick, first, we would need to compute $K(\mathbf{x}, \mathbf{x})$, $K(\mathbf{x}, \mathbf{x}')$ and $K(\mathbf{x}', \mathbf{x}')$. This is done only once, at the beginning of the algorithm. Second, as we implement Perceptron loops, we at least we have to compute $\alpha_1 y K(\mathbf{x}, \mathbf{x}) + \alpha_2 y' K(\mathbf{x}, \mathbf{x}')$ and $\alpha_1 y K(\mathbf{x}, \mathbf{x}') + \alpha_2 y' K(\mathbf{x}', \mathbf{x}')$ to decide on an update of the parameter $\boldsymbol{\theta}$. In total, how many computations would be involved in these steps?
4. (6 points) **Feedforward Neural Networks** In this problem, we analyze a simple neural network to understand its classification properties. Consider the neural network depicted below, with ReLU activation functions (denoted by f) on all neurons, and a softmax activation function in the output layer:

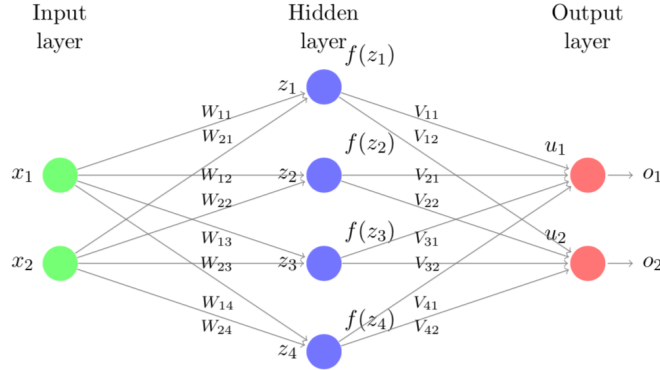


Figure 1: Simple NN for Problem 2

Given an input $\mathbf{x} = (x_1, x_2)^\top$, the hidden units in the network are activated in stages as described by the following equations:

$$\begin{aligned} z_1 &= x_1 W_{11} + x_2 W_{21} + W_{01}, & f(z_1) &= \max\{z_1, 0\}, \\ z_2 &= x_1 W_{12} + x_2 W_{22} + W_{02}, & f(z_2) &= \max\{z_2, 0\}, \\ z_3 &= x_1 W_{13} + x_2 W_{23} + W_{03}, & f(z_3) &= \max\{z_3, 0\}, \\ z_4 &= x_1 W_{14} + x_2 W_{24} + W_{04}, & f(z_4) &= \max\{z_4, 0\}, \end{aligned}$$

$$\begin{aligned} u_1 &= f(z_1)V_{11} + f(z_2)V_{21} + f(z_3)V_{31} + f(z_4)V_{41} + V_{01}, & f(u_1) &= \max\{u_1, 0\}, \\ u_2 &= f(z_1)V_{12} + f(z_2)V_{22} + f(z_3)V_{32} + f(z_4)V_{42} + V_{02}, & f(u_2) &= \max\{u_2, 0\}. \end{aligned}$$

The final output of the network is obtained by applying the softmax function to the last hidden layer,

$$\begin{aligned} o_1 &= \frac{e^{f(u_1)}}{e^{f(u_1)} + e^{f(u_2)}}, \\ o_2 &= \frac{e^{f(u_2)}}{e^{f(u_1)} + e^{f(u_2)}}. \end{aligned}$$

In this problem, consider the following parameter settings:

$$\begin{bmatrix} W_{11} & W_{21} & W_{01} \\ W_{12} & W_{22} & W_{02} \\ W_{13} & W_{23} & W_{03} \\ W_{14} & W_{24} & W_{04} \end{bmatrix} = \begin{bmatrix} 1 & 0 & -1 \\ 0 & 1 & -1 \\ -1 & 0 & -1 \\ 0 & -1 & -1 \end{bmatrix}$$

$$\begin{bmatrix} V_{11} & V_{21} & V_{31} & V_{41} & V_{01} \\ V_{12} & V_{22} & V_{32} & V_{42} & V_{02} \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 \\ -1 & -1 & -1 & -1 & 2 \end{bmatrix}$$

Answer the following:

- Consider the inputs $x_1 = 3$, $x_2 = 14$. What is the feedforward output $(o_1, o_2)^\top$ of the network?
- In this problem we visualize the “decision boundaries” in $\mathbf{x}$ -space, corresponding to the four hidden units. These are the lines in $\mathbf{x}$ -space where the values of z_1, z_2, z_3, z_4 are exactly zero. Plot the decision boundaries of the four hidden units using the parameters of W provided above. What is the area of the region that corresponds to the negative value for all hidden units?
- Using the same matrix V as above, what is the value of o_1 in the following three cases?:
 - Assuming that $f(z_1) + f(z_2) + f(z_3) + f(z_4) = 1$
 - Assuming that $f(z_1) + f(z_2) + f(z_3) + f(z_4) = 0$
 - Assuming that $f(z_1) + f(z_2) + f(z_3) + f(z_4) = 3$

Programming Problems (24 points)

Instructions and background:

In Programming Problems 1&2, we work with the UCI Sonar dataset (Connectionist Bench: Mines vs. Rocks), a binary classification dataset used to train models to distinguish between sonar signals bounced off metal cylinders and rocks. It contains 208 instances (in rows), each with 60 numeric features representing frequency band energy, and a target label (the last column). Depending on the readings, we would like to classify nearby objects into “M” (metallic objects) or into “R” (rocks). With this dataset you will practice the methods of Kernel Perceptron while employing the `cvs` module to import the required data.

In Problem 3, you will also train simple neural networks in `tensorflow`.

As with the last project, please use Python’s `NumPy` numerical library for handling arrays and array operations; use `matplotlib` for producing figures and plots, as well as the `scikit-learn` module. Along with the assignment, you will find the following three files:

- `main_HW7.py` which contains some code and comments to get you started.
- `utils.py`, where there are a few functions already defined for you to load the data.
- `sonar.all-data.csv`, which contains the UCI Sonar dataset. (You can also access it here: [here](#)).

- Kernel perceptron algorithm implementation** (10 points) You will implement a Kernel perceptron algorithm using a Gaussian Kernel. Recall that $K(\mathbf{x}, \mathbf{y}) = \exp\left(-\frac{\|\mathbf{x}-\mathbf{y}\|^2}{2\sigma^2}\right)$. Complete the following functions in `main_HW7.py`:

- Define the function `GKPA_pred(alpha,Xtrain,X,sigma)`, which makes a prediction of the label of the feature vector $\mathbf{x}$ (represented as $\mathbf{X}$) using the classifier defined through the output of the Kernel perceptron algorithm with a Gaussian Kernel of variance `sigma`.
- Define the function `GKPA_train(alpha0,Xtrain,ytrain,T,sigma)`, which implements the Kernel perceptron algorithm with a Gaussian Kernel to find or *train* or `alpha`. This function takes as input a feature matrix `Xtrain`, a vector of labels `ytrain` and `sigma`.

- (c) Run the functions `extract_features` and `extract_labels` in `utils.py` to obtain your feature matrix and label vector datasets. Using `sklearn` split your data into training and test datasets, `Xtrain`, `ytrain`, `Xtest`, `ytest`. (Employ the default value `random_seed = 0` for grading purposes).
 - (d) Using the function `accuracy` in `utils.py` pick the best `sigma` value from the set $\{0.1, 1, 10\}$. Explain the reason of your choice.
 - (e) Report the accuracy numbers on train and test datasets obtained using the best `sigma`, after running the outer loop for 10 iterations (i.e., $T = 10$).
2. **Comparison with a polynomial Perceptron algorithm** (10 points). Modify the functions of Problem 1 as follows:
- (a) `P_KPA_pred(alpha, Xtrain, X, p)`, which makes a prediction of the label of the feature vector $\mathbf{x}$ (represented as `X`) using the classifier defined through the output of the Kernel perceptron algorithm with a polynomial Kernel with power p .
 - (b) Define the function `P_KPA_train(alpha0, Xtrain, ytrain, T, p)`, which implements the Kernel perceptron algorithm with a polynomial Kernel to find or *train* or `alpha`. This function takes as input a feature matrix `Xtrain`, a vector of labels `ytrain` and `sigma`. (Employ the default value `random_seed = 0` for grading purposes).
 - (c) Run the functions `extract_features` and `extract_labels` in `utils.py` to obtain your feature matrix and label vector datasets. Using `sklearn` split your data into training and test datasets, `Xtrain`, `ytrain`, `Xtest`, `ytest`. (Use a random seed of 0 for grading purposes).
 - (d) Using the function `accuracy_p` in `utils.py` pick the best `p` value from the set $\{1, 2, 10\}$.
 - (e) Report the accuracy numbers on train and test datasets obtained using the best `sigma`, after running the outer loop for 100 iterations at most (i.e., $T = 100$).
 - (f) How do the results compare with those of Problem 1?
3. **Tensorflow** (4 points) In this problem, you will use the TensorFlow playground <http://playground.tensorflow.org>, which is a nice visual tool for training simple Multi Layer Perceptrons (MLPs). Your goal in this problem is to carefully select input features to design the “smallest” MLP classifiers that can achieve low test loss for each of the 4 datasets in TensorFlow playground. Figure 2 shows the four data sets available in the tensorflow playground. By “smallest” we mean having least number of neurons in the network. By low test loss we mean a test loss < 0.1 for the swiss roll dataset and a test loss of ≈ 0 for the rest of the datasets. Submit screenshots after your networks achieve the required test loss for each of the datasets. Attach this to your Homework 7P (Discussion) solution.

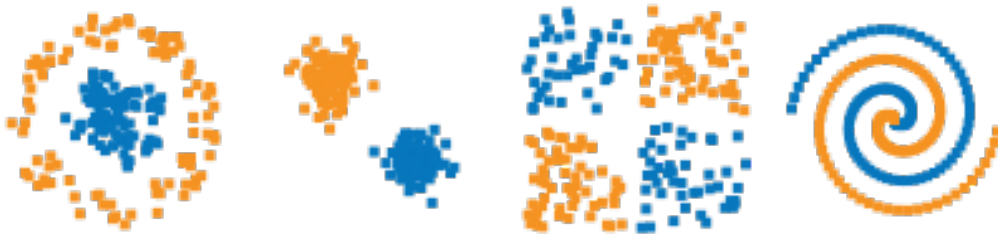


Figure 2: Toy data sets in tensorflow